

E: Efficient Encoding

Codificación eficiente

Nombre del archivo fuente (*Source file name*): `encoding.c`, `encoding.cpp`, `encoding.java` o `encoding.py`

Autor (*Author*): Rafael García

Es el año 3057. El Interplanetary Compression Priority Code (ICPC) tiene la tarea de mantener las comunicaciones entre las colonias humanas dispersas por la galaxia. Para sobrevivir en un entorno hostil con un ancho de banda muy limitado, cada mensaje debe comprimirse de la forma más eficiente posible antes de transmitirse.

Cada mensaje es un multiconjunto (*multiset*) de símbolos interplanetarios. El ICPC comprime mensajes escritos con un alfabeto de n símbolos, asignando a cada símbolo una palabra de código (*codeword*) construida mediante un código binario libre de prefijos (*binary prefix-free code*), es decir, ninguna palabra de código es prefijo de otra. Como de costumbre, el ICPC se apoya en estrategias de codificación óptimas que minimizan el número total de bits requeridos, tal como lo hacían en su momento los legendarios esquemas de compresión del siglo XXI.

Sin embargo, el número de ocurrencias de cada símbolo no es constante en el tiempo (en realidad, el ICPC trabaja con sus valores esperados). Estos fluctúan debido a tormentas solares, a la rotación del núcleo cuántico y a otros fenómenos cósmicos que solo los astrofísicos fingen entender. Para hacer frente a esta variabilidad, entre el tiempo 0 y una cota superior de tiempo fija T , el ICPC modela el valor esperado del número de ocurrencias de cada símbolo como una función lineal por tramos (*piecewise-linear function*) especificada por tres puntos con coordenadas en los números reales no negativos (es decir, en $\mathbb{R}_{\geq 0}$): $(0, a_i)$, (m_i, b_i) y (T, c_i) , con $0 < m_i < T$, y a_i , b_i y c_i enteros. Entre estos puntos, el valor esperado del número de ocurrencias del i -ésimo símbolo, $f_i(t)$, varía linealmente sobre los valores reales. En cualquier tiempo $t \in [0, T]$, el valor esperado del número de ocurrencias del símbolo i es:

$$f_i(t) = \begin{cases} a_i + \frac{b_i - a_i}{m_i} \cdot t, & 0 \leq t \leq m_i, \\ b_i + \frac{c_i - b_i}{T - m_i} \cdot (t - m_i), & m_i \leq t \leq T. \end{cases}$$

Para elegir el mejor tiempo de codificación, el ICPC considera, para cada $t \in [0, T]$, el código binario libre de prefijos óptimo para las frecuencias $\{f_i(t)\}_{i=1}^n$ y evalúa su costo total

$$C(t) = \sum_{i=1}^n f_i(t) \ell_i(t),$$

donde $\ell_i(t)$ es la longitud de la palabra de código asignada al símbolo i por un código óptimo en el tiempo t .

Tu tarea es ayudar al ICPC a elegir el tiempo de codificación ideal: encuentra el mínimo tiempo $t \in [0, T]$ que minimiza el costo total de codificación $C(t)$.

Entrada (Input)

La primera línea contiene el número de casos de prueba K ($1 \leq K \leq 10^3$). Para cada caso de prueba, la primera línea contiene dos enteros n y T ($1 \leq n \leq 100$ y $1 < T \leq 10^6$), que denotan el número de símbolos y el valor máximo del tiempo, respectivamente. Cada una de las siguientes n líneas contiene cuatro enteros a_i , m_i , b_i y c_i ($0 < m_i < T$ y $0 \leq a_i, b_i, c_i \leq 10^6$), que describen la función de frecuencia $f_i(t)$.

La entrada debe leerse de la entrada estándar.

Salida (Output)

Para cada caso de prueba, imprime en una línea dos números separados por un espacio: el menor tiempo entero $t \in [0, T]$ que minimiza $C(t)$, seguido del costo mínimo $C(t)$ redondeado a tres dígitos después del punto decimal.

La salida debe escribirse en la salida estándar.

Ejemplo de entrada (Sample Input)	Ejemplo de salida (Sample Output)
2 3 10 2 5 10 4 4 6 9 3 3 8 8 7 1 2 2 1 1 2	0 14.000 1 1.000